

Link to the source code: [Github Repository](#)

Authors

- Wojciech Bogacz 156034
- Krzysztof Skrobala 156039

Problem Description

The problem considered in this project involves a set of nodes, each represented by three integer values: two coordinates (x,y)(X, y)(X,y) that define the node's position in a two-dimensional plane, and a cost value associated with the node. The objective is to select exactly 50% of all available nodes (if the total number of nodes is odd, the number of selected nodes is rounded up) and construct a Hamiltonian cycle—that is, a closed path visiting each selected node exactly once and returning to the starting node. The goal is to minimize the sum of two components: 1. The total length of the constructed cycle, and 2. The total cost of all selected nodes The distances between nodes are calculated using the Euclidean metric, rounded to the nearest integer.

Implemented Algorithms

```
Operator1(parent1, parent2)
n = length(parent1)
targetSize = n / 2

edges1 = empty set
edges2 = empty set

for i = 0 TO n-1 do
    a = parent1[i]
    b = parent1[(i+1) mod n]
    add (a, b) to edges1

for i = 0 TO n-1 do
    a = parent2[i]
    b = parent2[(i+1) mod n]
    add (a, b) to edges2

visited = empty set
subpaths = empty list of lists

for i = 0 TO n-1 do
    start = parent1[i]

    if start in visited then
        CONTINUE

    path = { start }
    mark start as visited
    current = start

    while TRUE do
        nextIndex = index of current in parent1 + 1

        if nextIndex ≥ n then
            break

        next = parent1[nextIndex]
        edge = (current, next)

        if edge in edges1 AND edge in edges2 AND next not in
visited then
            append next to path
            mark next as visited
            current = next
        else
            break

    append path to subpaths

allNodes = set of all nodes in parent1
remove all visited nodes from allNodes

currentSize = total number of nodes in subpaths

while currentSize < targetSize AND allNodes is not empty do
    node = randomly selected element from allNodes
    append [node] to subpaths
    remove node from allNodes
    currentSize = currentSize + 1

randomly shuffle subpaths

offspring = empty list

for each path in subpaths do
    if random number < 0.5 then
        reverse path
    append path to offspring

RETURN offspring

GreedyRegretStart(instance, start_node):
    Select start_node
    Select second node with minimum (distance + cost)

    while selected nodes < K:
        for each unselected node v:
            compute best insertion cost
            compute second-best insertion cost
            regret = second_best - best
            score = regret - best_total_cost

        choose node with maximum score
        insert node at its best position

    return tour, inSelected[]

CleanPopulation(instance, population, targetSize):
    RemoveDuplicates
    if population too large:
        RemoveWorst
    if population too small:
        generate new LS-improved random solutions
    return population

GenerateInitialPopulation(instance, size):
    population = empty list
    while population size < size:
        start_node = random node
        initialization_id = choice from (regret, alfa_beta,
nn_anywhere, nn_end)
        if initialization_id == regret:
            solution, inSelected = GreedyRegret(instance, start_node)
        else if initialization_id == alfa_beta:
            alpha, beta = random values
            solution, inSelected = AlfaBeta(start_node, instance,
alpha=1, beta=1)
        else if initialization_id == nn_anywhere:
            solution, inSelected = NearestNeighborAnywhere(start_node,
instance)
        else if initialization_id == nn_end:
            solution, inSelected = NearestNeighborEnd(start_node,
instance)

        population.append(LocalSearch(instance, solution, inSelected))

    remove_dupliacates(population)

Main Loop

RunMethod(instance):
    population = GenerateInitialPopulation(instance, populationSize)

    start timer
    while time limit not exceeded:
        select random pairs of parents
        for each pair:
            offspring = crossover operator
            apply local search to offspring
            add offspring to population

        CleanPopulation(instance, population, populationSize)

    return best solution found
```

Results

Comparison of previous methods results

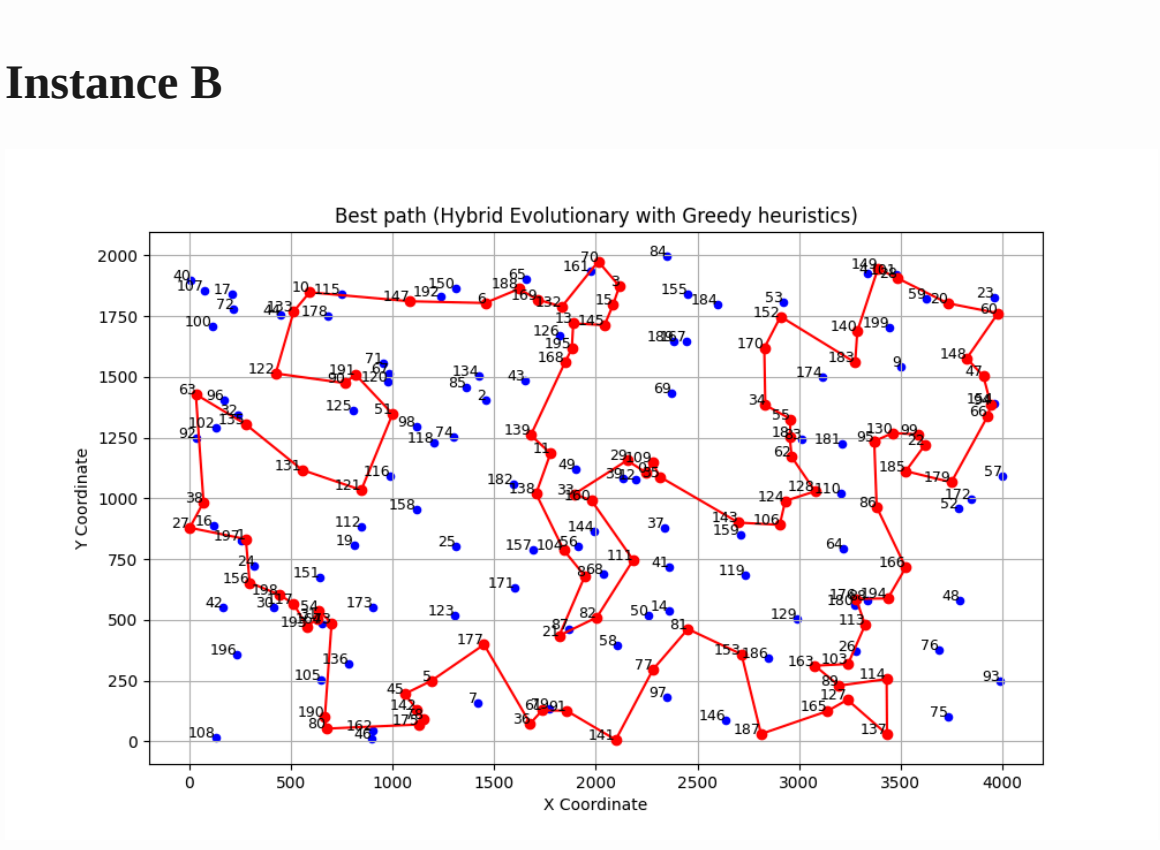
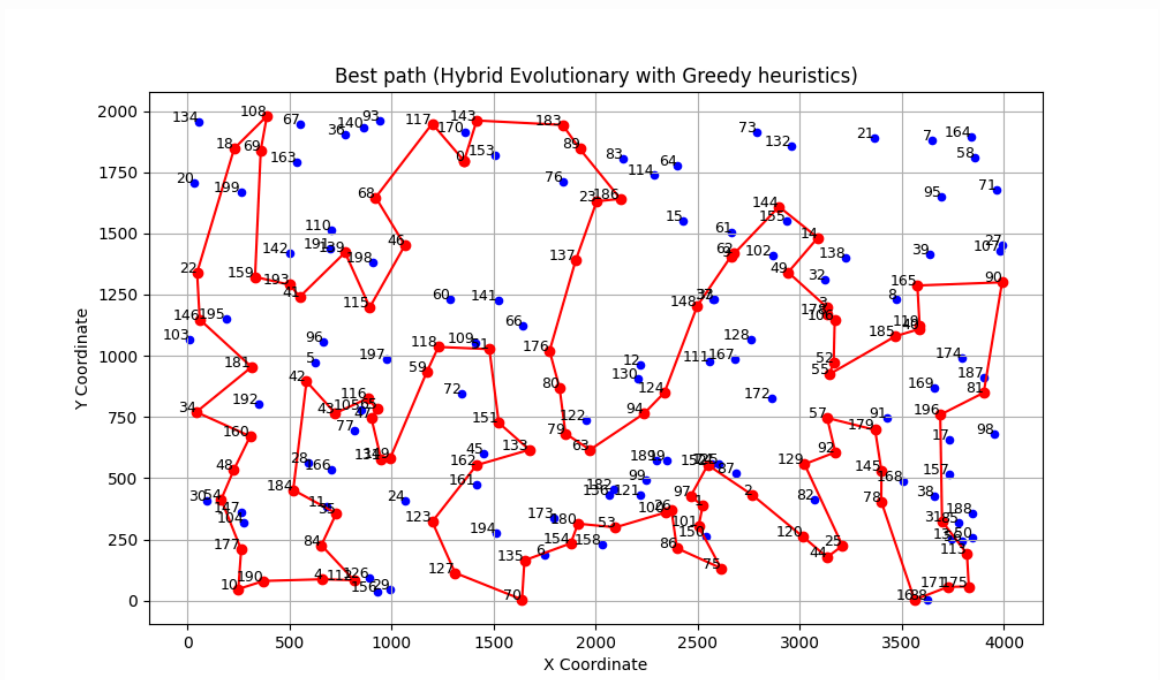
Instance A				
	Method	Best	Worst	Average
random		235308	292123	264677.17
nn_end		89198	123123	104012.785
nn_anywhere		71488	74410	72635.72
greedy cycle		72639	72639	72639.0
2-Regret Insertion		105852	123428	115474.93
Weighted Sum (α=1.00, β=1.00)		71108	73438	72130.85
greedy_intra:edges_start:greedy		70663	72648	71594.055
greedy_intra:edges_start:random		71564	76328	73528.81
greedy_intra:nodes_start:greedy		70687	73027	71739.395
greedy_intra:nodes_start:random		79677	99035	86909.13
steepest_intra:edges_start:greedy		70510	72614	71463.08
steepest_intra:edges_start:random		71246	78153	73699.995
steepest_intra:nodes_start:greedy		70626	73004	71615.93
steepest_intra:nodes_start:random		81322	95342	87939.335
steepest_intra:edges_start:random		71246	78153	73699.995
steepest_intra:edges_start:random with Cand mech.		73076	85487	77866.91
steepest_lm_intra:edges_start:random		71265	78247	73856.275
ILS_intra:edges_start:random		70901	71862	71303.60
steepest_multi_start_intra:edges_start:random		71149	72794	71299.05
LNS_no_search_intra:edges_start:random		69732	71868	70604.2
LNS_with_search_intra:edges_start:random		69397	71604	70453.9
Operator1		69875	70843	70518.45
Operator2		70154	72087	71630.65
Operator2_without_LS		70233	71965	71397.2

Hybrid evolutionary algorithm				
	Method	best	worst	average
GeneticWithHeuristics		69620	70145	69853.05

Instance B				
	Method	Best	Worst	Average
random		193234	234798	214279.235
nn_end		62606	77453	69764.43
nn_anywhere		49001	57324	51400.905
greedy cycle		50243	50243	50243.0
2-Regret Insertion		66505	77072	72454.77
Weighted Sum (α=1.00, β=1.00)		47144	55700	50918.82
greedy_intra:edges_start:greedy		46178	55471	50172.67
greedy_intra:edges_start:random		45537	51687	48154.865
greedy_intra:nodes_start:greedy		46373	55385	50347.525
greedy_intra:nodes_start:random		53417	71474	61453.295
steepest_intra:edges_start:greedy		45867	54814	49907.205
steepest_intra:edges_start:random		45903	51416	48195.845
steepest_intra:nodes_start:greedy		46371	55385	50201.47
steepest_intra:nodes_start:random		55686	71546	62955.885
steepest_intra:edges_start:random		45903	51416	48195.845
steepest_intra:edges_start:random with Cand. Mech.		45798	52670	48474.64
steepest_lm_intra:edges_start:random		45941	51587	48214.715
ILS_intra:edges_start:random		45320	46753	45463.3
steepest_multi_start_intra:edges_start:random		45405	46222	45767.4
LNS_no_search_intra:edges_start:random		44415	46341	45336.95
LNS_with_search_intra:edges_start:random		44012	45376	44511.30
Operator1		43924	45040	44757.4
Operator2		44244	45547	44894.6
Operator2_without_LS		44268	45724	44679.9

Hybrid evolutionary algorithm				
	Method	best	worst	average
GeneticWithHeuristics		43828	44907	44327.6

Paths visualization



Time

Instance A				
	Method	min	max	average
greedy_intra:edges_start:greedy		0.00	0.006	0.003
greedy_intra:edges_start:random		0.062	0.099	0.080
greedy_intra:nodes_start:greedy		0.000	0.005	0.002
greedy_intra:nodes_start:random		0.066	0.118	0.085
steepest_intra:edges_start:greedy		0.001	0.005	0.002
steepest_intra:nodes_start:greedy		0.000	0.005	0.002
steepest_intra:nodes_start:random		0.049	0.142	0.071
steepest_intra:edges_start:random		0.039	0.050	0.045
steepest_intra:edges_start:random with Can. Mech.		0.008	0.019	0.011
steepest_lm_intra:edges_start:random		0.025	0.049	0.030
ILS_intra:edges_start:random		4.805	4.815	4.811
steepest_multi_start_intra:edges_start:random		4.443	5.99	4.805
LNS_no_search_intra:edges_start:random		4.810	4.891	4.840
LNS_with_search_intra:edges_start:random		4.812	4.881	4.832

New Methods Results				
	Method	min	max	average
GeneticWithHeuristics		-	-	as MSLS

Instance B				
	Method	min	max	average
greedy_intra:edges_start:greedy		0.0	0.01	0.004
greedy_intra:edges_start:random		0.064	0.102	0.081
greedy_intra:nodes_start:greedy		0.0	0.007	0.003
greedy_intra:nodes_start:random		0.058	0.107	0.082
steepest_intra:edges_start:greedy		0.001	0.01	0.004
steepest_intra:nodes_start:greedy		0.001	0.007	0.004
steepest_intra:nodes_start:random		0.048	0.086	0.063
steepest_intra:edges_start:random		0.038	0.054	0.045
steepest_intra:edges_start:random with CAn. Mech.		0.009	0.05	0.012
steepest_lm_intra:edges_start:random		0.020	0.053	0.035
ILS_intra:edges_start:random		4.499	4.515	4.503
steepest_multi_start_intra:edges_start:random		4.377	5.077	4.498
LNS_no_search_intra:edges_start:random		4.499	4.609	4.554
LNS_with_search_intra:edges_start:random		4.501	4.607	4.540

	Method	min	max	average
GeneticWithHeuristics		-	-	as MSLS

Conclusions

In this assignment we explored influence of greedy regret initialization on the evolutionary algorithm, as it performed the best among all approaches. The results indicate that the hybrid evolutionary algorithm with greedy regret initialization outperforms other methods in terms of solution quality for both instances A and B. The best solutions obtained by this method are lower than those achieved by the other approaches, demonstrating its effectiveness in solving the problem at hand.